KDE Connect WebApp

Bringing cross-device integration to the browser with a lightweight, embedded server architecture

Vaisakh K M
202117b3543

Project Overview

What is KDE Connect?

- Popular **open-source application for seamless workflow integration across devices**. Features include clipboard sync, notification mirroring, remote device control, and file sharing.
- Current platforms: Qt-based desktop clients, Android, and iOS.

The Gap

- No web client exists, limiting accessibility for users who cannot install native applications on every device.
- My **solution brings KDE Connect capabilities directly to the browser**, requiring installation on just one device.

Core Objective

Protocol Porting

Adapt KDE Connect protocols for web environments, including custom TLS implementation for secure communication

Lightweight Server

Develop a Kotlin-based server that runs embedded within resource-constrained Android environments

Plugin Architecture

Design server as a plugin for native apps, enabling seamless integration with existing KDE Connect installations

Web Client

Build a responsive web interface leveraging modern browser APIs to deliver core KDE Connect features

Technical Architecture

Native App

KDE Connect on Android/Desktop with embedded server plugin

Server Layer

Kotlin server handling protocol translation and TLS encryption

Web Client

Browser-based interface accessing features via APIs

Platform Challenges & Solutions

Browser API Limitations

Challenge: Traditional web APIs lack native file system access and persistent connections.

Solution: Leverage FileSystemAccess API for file sync and WebSocket connections for real-time communication.

Android API Restrictions

Challenge: Android 12+ prevents background clipboard access for security reasons.

Solution: Implement foreground-aware sync mechanisms with user-initiated clipboard operations and notification-based workflows.

...Platform Challenges & Solutions

Resource Constraints

Challenge: Server must run efficiently on mobile devices with limited CPU and battery.
Solution: Optimize Kotlin server with minimal memory footprint, efficient connection pooling, and adaptive performance tuning.

Secure Context Requirement

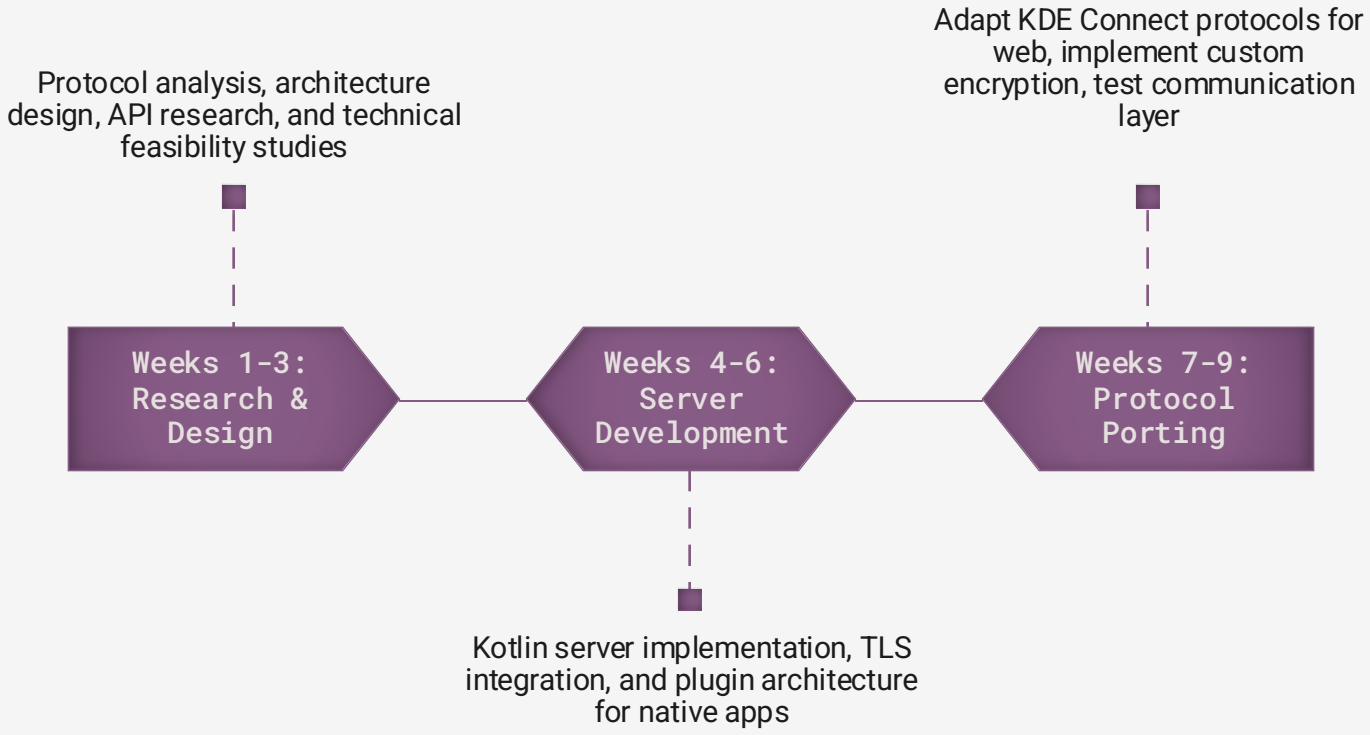
Challenge: Sensitive APIs like WebRTC and File System Access require HTTPS, blocking use on plain HTTP.
Solution: Use HTTPS with a self-signed certificate and enable trust through QR-code–based certificate sharing.

Planned Features

Remote Camera Future	Live Screen Mirroring Future	Full Remote Control Future	Remote Lock Future	Remote Wipe Future	Remote over Internet Future
Web Integration Future	SFTP Filesystem Future	Virtual Monitor Future	Run Remote Commands Future	Phone → PC Keyboard KDE Connect	Phone as Touchpad KDE Connect
Presenter / Laser Pointer KDE Connect	Pause PC Media on Call KDE Connect	Reply SMS from PC KDE Connect	Two-way Notifications KDE Connect	Shared Clipboard KDE Connect	Control PC Media Player KDE Connect
End-to-End Encryption KDE Connect	Zero-config LAN only KDE Connect	Send/Receive Files KDE Connect	SMS from Desktop KDE Connect	Find My Phone (ring) KDE Connect	Battery Monitor KDE Connect
Signal Strength KDE Connect	Phone Notifications → PC KDE Connect	Incoming Call Notification KDE Connect	Contacts Sync/View KDE Connect	Browse Phone Files KDE Connect	Remote Volume Control KDE Connect

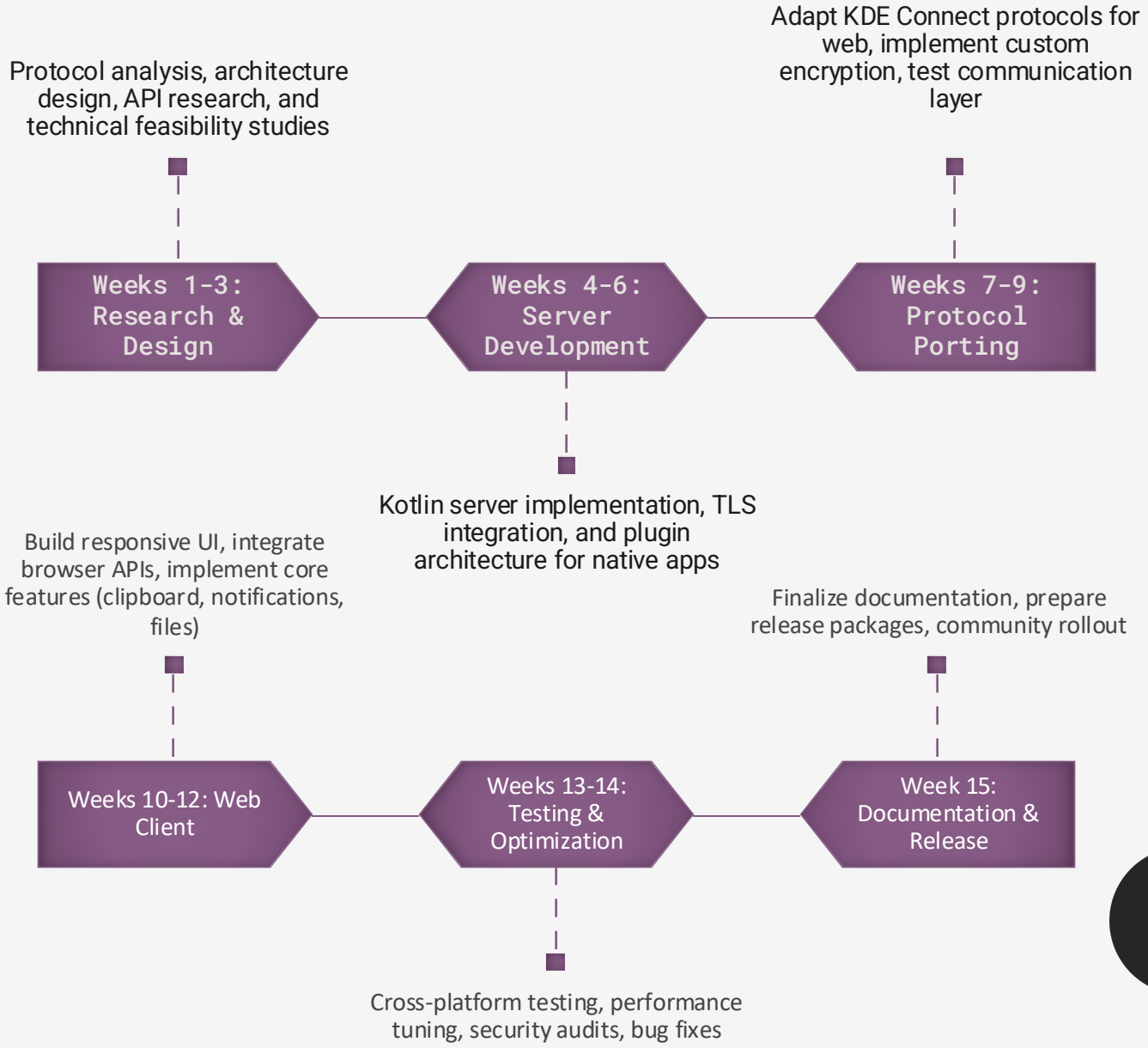
Timeline

15 week development timeline for the project



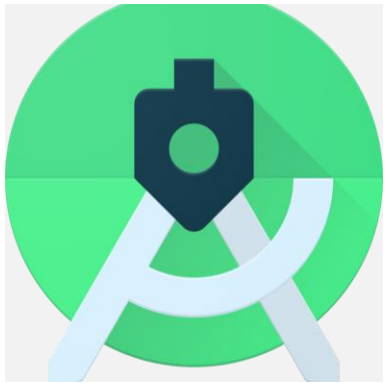
Timeline

15 week development timeline for the project



Tools and Technologies

KDE Connect



Android Studio

Ktor/nanohttpd

Java/Kotlin

Git

Svelte

Thank you